

A satellite view of Earth at night, showing the curvature of the planet and numerous glowing city lights across the continents. The text is overlaid on this image.

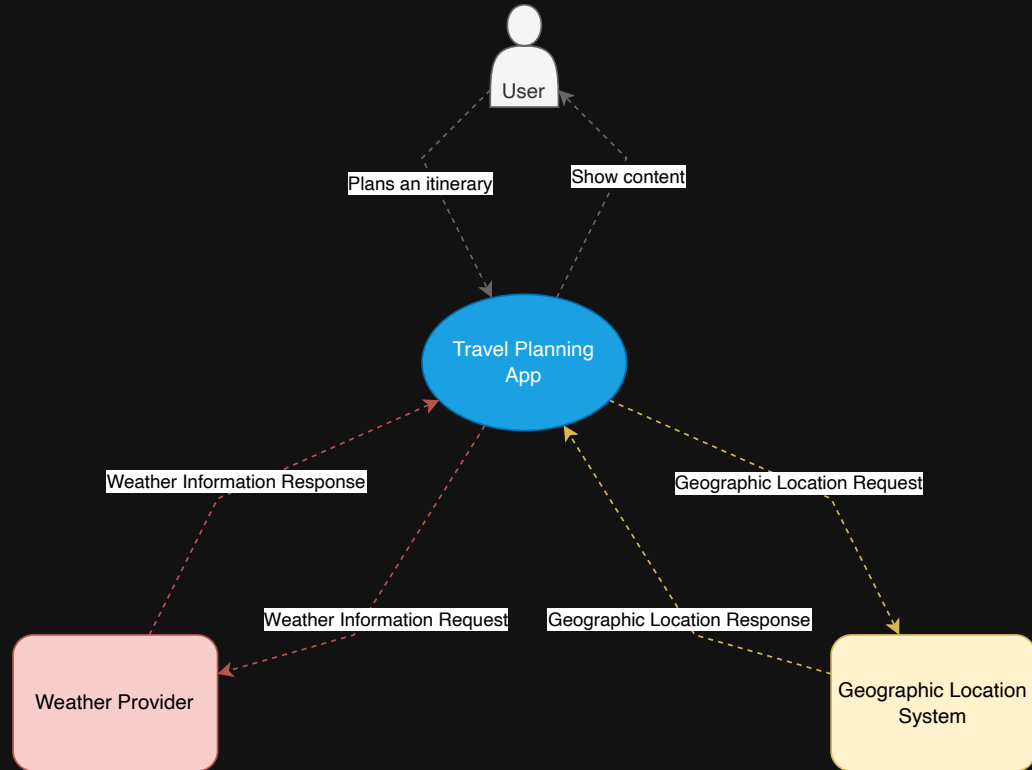
CloudAppDev

Milestone 2: Microservices & Kubernetes

From Monolith to Distributed Architecture

Simon Driescher, Samuel Behrmann, Simon Blaser

System Context



System Context & Features

Core Features

-  **User Management** - Registration, profiles, avatars
-  **Itinerary Management** - CRUD operations, multi-location trips
-  **Search & Discovery** - Find trips by destination, keywords
-  **Social Interaction** - Likes, comments, engagement
-  ***NEW* Interactive Maps** - Leaflet integration, location selection
-  ***NEW* Personalized Newsletter** - Interest-based recommendations
-  ***NEW* Weather Insights** - Forecasts, warnings, travel advice

Evolution: Monolith → Microservices

Milestone 1 (Monolith)

- Next.js Fullstack Application
- PostgreSQL + MongoDB
- Cloud Run (PaaS)
- Compute Engine (IaaS)

Milestone 2 (Microservices)

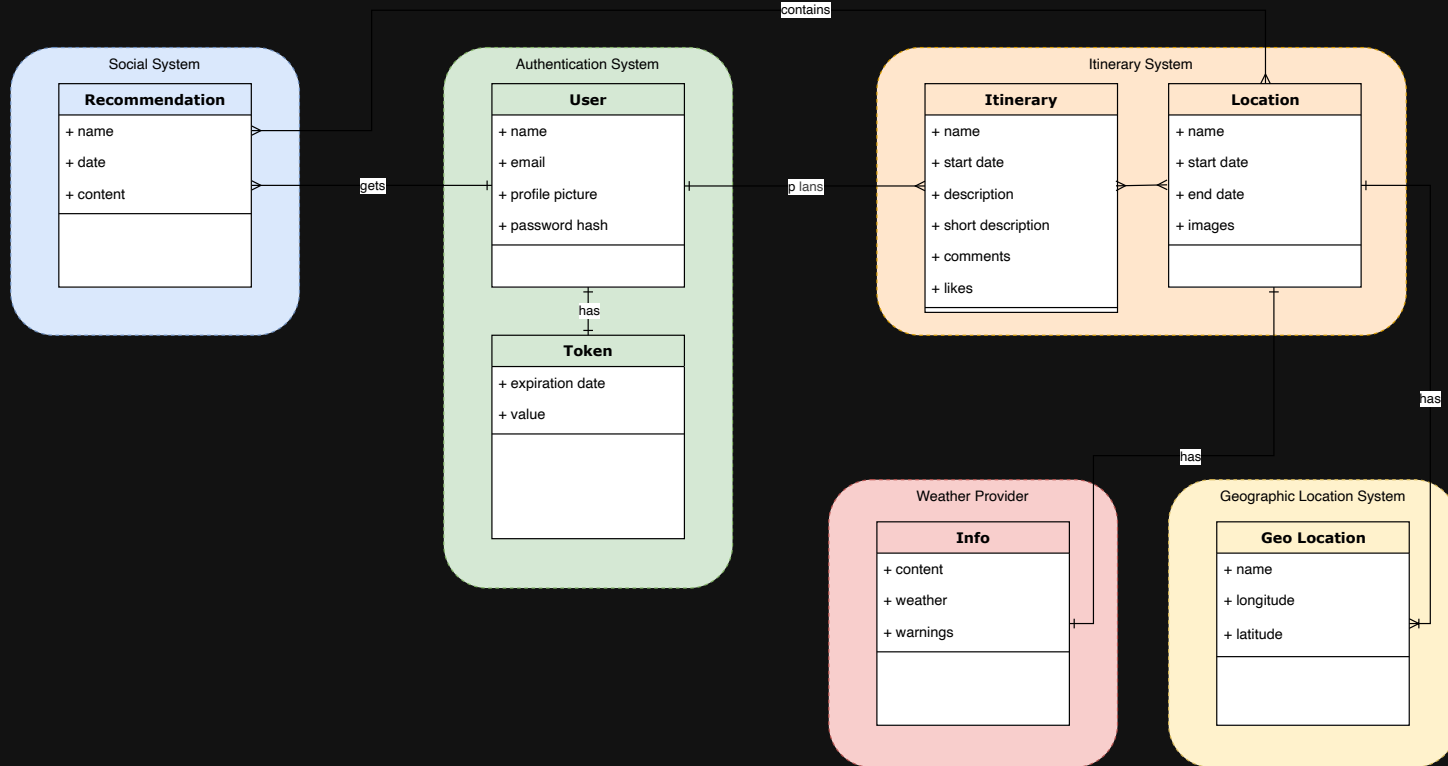
- 5 specific services
- PostgreSQL (User + Itinerary)
- MongoDB (Social Service)
- Google Kubernetes Engine (GKE)



Microservices Architecture

Domain-Driven Design with clear Service-Boundaries

Domain Model



Repository Structure

Monorepo Organization

```
/app          - Next.js Frontend
/services     - Backend Microservices
├── user-service
├── itinerary-service
├── social-service
├── travel-info-service
└── seeder
/nginx        - API Gateway Config
/k8s          - Kubernetes Manifests
/terraform    - IaC for GCP
/locust       - Performance Tests
/seed-data    - Initial Dataset
```

Technology Stack

Frontend:

- Next.js 15.5.4 + React 19.1.0
- PrimeReact 10.9.7 UI Components
- Tailwind CSS 4

Backend:

- NestJs
- Prisma 6.19.0 ORM
- Mongoose 8.8.4 ODM

External Services:

- SendGrid Email API
- Weather APIs

Service Übersicht

User Service (Port 8080)

- User Registration & Management
- NestJS + Prisma + PostgreSQL

Itinerary Service (Port 8081)

- Travel Route CRUD Operations
- NestJS + Prisma + PostgreSQL

Social Service (Port 8082)

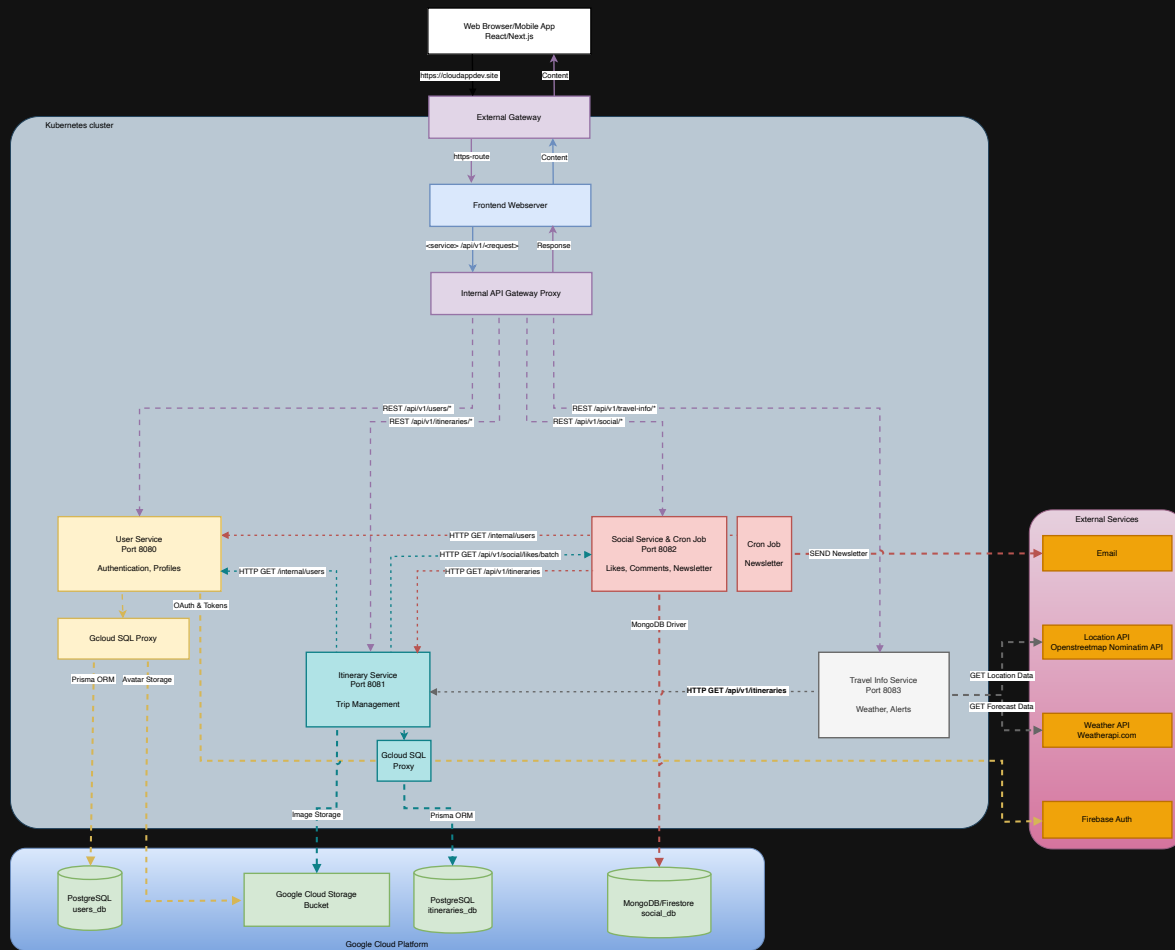
- Likes & Comments System
- Newsletter Subscriptions
- NestJS + Mongoose + MongoDB

Travel Info Service (Port 8083)

- Weather Data Integration
- Travel Warnings & Safety Info
- NestJS + Axios

API Gateway (Nginx Port 80)

Routing, Load Balancing, 50MB Upload Support, TLS Termination



API Gateway Configuration

```
# Upstream Service Definitions
upstream user_service {
    server user-service:8080;
}
upstream itinerary_service {
    server itinerary-service:8081;
}
upstream social_service {
    server social-service:8082;
}
upstream travel_info_service {
    server travel-info-service:8083;
}
server {
    listen 8000;
    location /api/v1/users {
        proxy_pass http://user_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /api/v1/itineraries {
```

Features

- Single endpoint for the frontend
- Lightweight proxy

Performance Testing

Locust Framework mit realistischen Workloads

Test Setup & Data Initialization

Seed Data

Database State:

- 10 diverse users (Emma, Liam, Sofia, ...)
- 18 itineraries across 6 continents
- 4 comments, 11 likes
- Trending: Rome Colosseum, Tokyo Tech, Vienna

Coverage:

- Rome, Barcelona, Tokyo, Kyoto
- Iceland, Norway, Bangkok, Mumbai
- New Zealand, Australia, Russia
- Switzerland, Vienna, Paris, Costa Rica

User Journeys

Casual Browsers (50%)

- Quick browse (5x)
- Search destinations (3x)
- View popular itineraries (2x)
- Optional registration (1x)

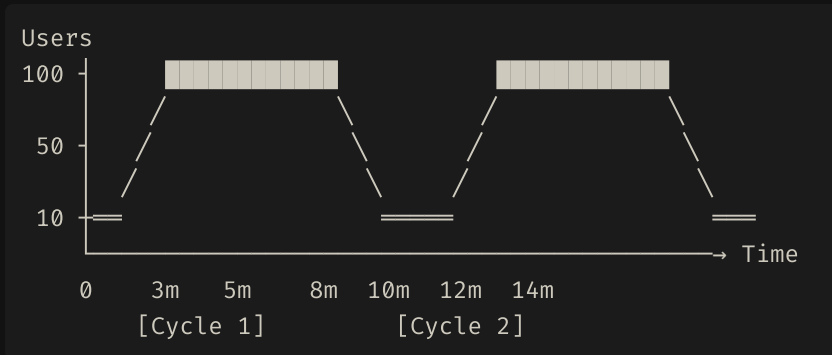
Active Users (30%)

- Browse & like (4x)
- Check own itineraries (3x)
- Create new route (2-4 locations)
- Comment on posts (3x)

New Users (20%)

Scenario A: 100 Peak / 10 Low Users

Load Pattern



Duration: ~14 minutes (2 cycles)

- Low: 2min @ 10 users
- Ramp up: 1min → 100 users
- Peak: 3min @ 100 users
- Ramp down: 1min → 10 users

Performance Metrics

Throughput: 60-80 req/sec

Response Times:

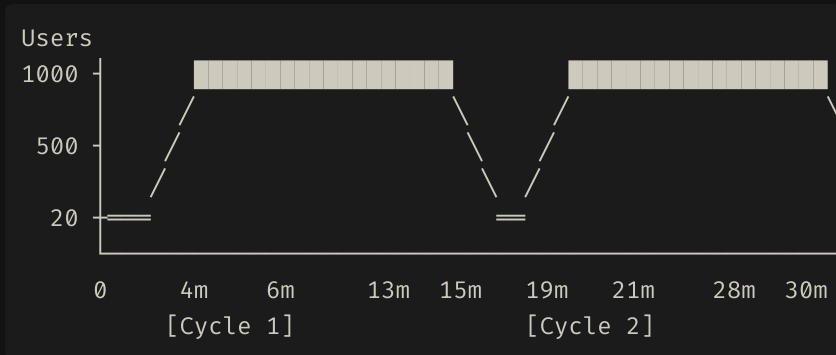
- p95 < 800ms @ peak
- p99 < 1500ms
- Error rate < 0.5%

Resource Utilization:

- User Service: CPU 15-25%, Mem 100MB
- Itinerary Service: CPU 20-35%, Mem 150MB
- Social Service: CPU 10-15%, Mem 80MB
- PostgreSQL: CPU 30-40%
- MongoDB: < 100ms queries

Scenario B: 1000 Peak / 20 Low Users

Load Pattern



Duration: ~30 minutes (2 cycles)

- Low: 4min @ 20 users
- Ramp up: 2min → 1000 users
- Peak: 7min @ 1000 users
- Ramp down: 2min → 20 users

Performance Metrics

Throughput: 600-800 req/sec

Response Times:

- p95: 1500-2000ms (3x degradation)
- p99: 3000ms+
- Error rate: 0.5-2% @ peak

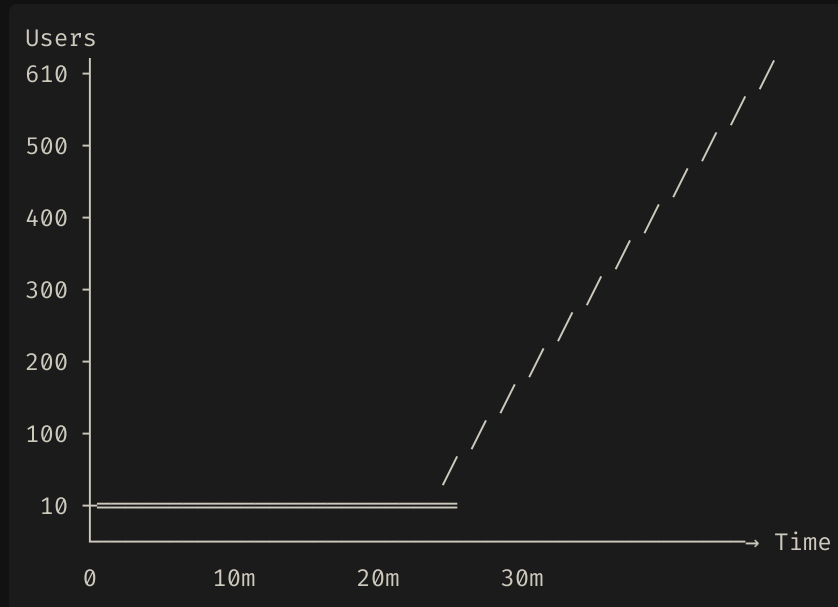
Bottleneck: PostgreSQL Connection Pool

- 20 connections/service (40 total)
- Queue saturation @ 450+ users
- 500-2000ms delays on simple queries

Mitigation Strategy:

Once-in-a-Lifetime Workload

Continuous Growth Pattern



Growth: 20 users/minute **Formula:** $\text{users} = 10 + (20 \times \text{minutes})$ **Max cap:** 3000 users (safety)

Breaking Points

Users	Req/sec	p95	Error	State
50	60	200ms	<0.1%	Green
150	180	400ms	<0.5%	Green
250	300	1000ms	1.5%	Yellow
350	420	1500ms	3%	Red
450	540	2000ms	5%	Critical
550+	660+	3000ms+	10%+	Failed

Performance Thresholds

No Degradation

Load: ~200 users (240 req/sec)

Criteria: p95 < 500ms, error < 1%

PostgreSQL: CPU < 40%

Status: All services healthy

With Degradation

Load: 200-450 users (240-540 req/sec)

Criteria: p95 < 2000ms, error < 5%

Connection Pool: 80%+ utilized

Status: Functional but slower

Failure

Load: 450+ users (540+ req/sec)

Criteria: p95 ≥ 2000ms OR error ≥ 5%

Connection Pool: Exhausted

Status: Cascading timeouts

System exhibits **graceful degradation** – casual browsing degrades first, existing reads remain responsive

Bottleneck Analysis

Primary: PostgreSQL Connection Pool

Problem:

- 20 connections per service (40 total)
- Queue saturation @ 450+ users
- 500-2000ms delays vs. 10-50ms baseline

Impact:

- Itinerary creation slows significantly
- Registration fails (1-2% error rate)
- Cascading timeouts across services

Solution:

```
# PgBouncer Deployment
apiVersion: apps/v1
```

Recommended Optimizations

1. Connection Pooling

- Deploy PgBouncer (200+ connections)
- Session-based pooling mode

2. Read-Write Splitting

- Route reads to replicas
- Writes to primary only

3. Cache Layer

- Redis for popular itineraries
- 60-second TTL

4. Circuit Breaker

Runtime Overview

Live Application: CloudAppDev.site

GCP Project: oceanic-citadel-474512-c1

Infrastructure Links



[GKE Workloads](#)



[GKE Gateways](#)



[Cloud SQL Instances](#)



[Firestore Databases](#)

Architecture Highlights

- **5 Microservices** on GKE Autopilot
- **2 PostgreSQL** instances (Users, Itineraries)
- **1 MongoDB** instance (Social data)
- **Nginx Gateway** for routing
- **Google Cloud Storage** for images
- **SendGrid API** for newsletters

Deployment Automation

CI/CD Pipeline

```
# Build & Push Container Images
docker build -t gcr.io/$PROJECT/user-service \
  ./services/user-service
docker push gcr.io/$PROJECT/user-service

# Deploy to Kubernetes
kubectl apply -f k8s/app-deployment.yaml
kubectl apply -f k8s/app-service.yaml
kubectl apply -f k8s/gateway.yaml

# Verify Deployment
kubectl rollout status deployment/user-service
kubectl get pods -l app=user-service
```

Rolling Updates:

- Zero-downtime deployment
- Gradual replica replacement
- Automatic rollback on failure

Health Checks

```
# Liveness & Readiness Probes
apiVersion: apps/v1
kind: Deployment
spec:
  template:
    spec:
      containers:
        - name: user-service
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10

          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 5
```

DevOps: Infrastructure as Code

Terraform (GCP Resources)

Managed Services:

- Cloud SQL (PostgreSQL 16)
- Firestore (NoSQL)
- Cloud Storage Buckets
- Service Accounts & IAM Roles

```
resource "google_sql_database_instance" "users" {  
  database_version = "POSTGRES_16"  
  region          = "europe-west1"  
  
  settings {  
    tier = "db-custom-2-8192"  
    backup_configuration {  
      enabled = true  
    }  
  }  
}
```

Kubernetes

Cluster Resources:

- External & Internal Gateways
- HTTP Routes
- Service Health Checks
- GKE Service Accounts

Per Microservice:

1. Deployment (service code)
2. Kubernetes Service (networking)
3. Horizontal Pod Autoscaler
4. Secrets (env vars, keys, tokens)

Key Learnings

Architectural Benefits

✓ Service Isolation

- Independent deployment & scaling
- Technology flexibility per service
- Fault isolation (cascading failures prevented)

✓ Scalability

- Horizontal pod autoscaling
- Targeted resource allocation
- Cost-efficient compute usage

✓ Maintainability

- Clear domain boundaries

Operational Complexity

⚠ Distributed System Challenges

- Network latency overhead
- Service discovery requirements
- Complex debugging across services

⚠ Database Bottlenecks

- Connection pool exhaustion @ 450+ users
- Requires PgBouncer for production
- Read-write splitting needed

⚠ Infrastructure Management

- Kubernetes learning curve

Vielen Dank!

CloudAppDev Milestone 2

Microservices Architecture | Google Kubernetes Engine | Locust Performance Testing
Express.js + Prisma | PostgreSQL 16 + MongoDB | Nginx API Gateway